



CS205 C/C++ Programming - Project 5: GPU Acceleration with CUDA

Name: 殷海洲(Yin Haizhou)

SID: 12210226

Part 1 - Analysis

Requirement

1. Implement the expression $B = aA + b$, where a and b are scalars, A and B are matrices of the same size. You can implement it by modifying the example [matadd.cu](#).
2. Compare the matrix multiplication by OpenBLAS on CPU with that by cuBLAS on GPU. cuBLAS is a GPU-accelerated library provided by NVIDIA. Do not use some huge matrices, and 4096×4096 should be large enough.
3. (Optional) Something interesting on GPU.

Requirement Analysis

Following the requirements, the project 5 is only restricted to implement a basic scalar-matrix calculation and matrices multiplication using OpenBLAS and cuBLAS.

In [matadd.cu](#) example, matrices are built in structure of C, whereas C structure may not be very suitable for students who has learned classes for multiple lessons. As a result, Matrix class to implement the calculation requirements above is mainly considered.

Considering that CUDA of NVIDIA used to accelerate the program with invoking GPU instead of CPU is implemented, We may use server already equipped with CUDA which is provided in class or install one on my own computer. To make comparisons between effects of program on server and PC, both choices are to be adopted.

Finally, program will be run on different platforms and time of doing matrix multiplication will be recorded and compared.



1.1 An Insight into CUDA

What Is CUDA? CUDA is a parallel compute engine and software environments for GPU developed by NVIDIA, while GPU provides more cores but less cache compared with CPU. CUDA provides programmers an environment similar with C/C++ but supports parallel computation in GPU.

1.2 An Insight into cuBLAS

NVIDIA cuBLAS is a GPU-accelerated library for accelerating AI and HPC applications. In our project, we use cuBLAS to calculate complex and massive matrix multiplication together with using OpenBLAS, a matrix multiplication centered library.

1.3 An Insight into `matadd.cu`

As our first implementation using CUDA, we must be careful and class example will be a very good reference.

Initially, a matrix structure is defined with four elements: numbers of rows and columns, address of data of the matrix for CPU and GPU.

```
typedef struct
{
    size_t rows;
    size_t cols;
    float * data; // CPU memory
    float * data_device; //GPU memory
} Matrix;
```

This is for the reason that different ways to allocate memory are used for CPU computation and GPU computation. They own the same data but share no same memory for data while one is in CPU and another in GPU.

Then, each entry of the matrix are set the same value via `set()` .

Despite that the program for CPU are ordinarily coded, the most important part is to understand parallel computation by CUDA is realized using kernel as following.

```
__global__ void addKernel(const float * input1,
const float * input2, float * output, size_t len)
{
    int i = blockDim.x * blockIdx.x + threadIdx.x;
    if(i < len)
        output[i] = input1[i] + input2[i];
}
```

Here, `__global__` indicates the kernel function run on GPU devices and can be called by CPU. The first line index the present thread and then add two entries while preventing exceeding its memory. From this significant clause, it is very clear how to apply our functions.

Eventually but also for the first line of [matadd.cu](#), a macro is defined to calculate the time cost, which is convenient and efficient.

Part 2 - Code

2.1 What do we need?

```
#include <iostream>
#include <sys/time.h>
#include <cblas.h>
#include <cublas_v2.h>
#include "cudamatrixclass.h"

using namespace std;
```

Also we use the same method to count time.

```
#define TIME_START gettimeofday(&t_start, NULL);
#define TIME_END(name) gettimeofday(&t_end, NULL); \
    elapsedTime = (t_end.tv_sec - \
t_start.tv_sec) * 1000.0; \
```

```

elapsedTime += (t_end.tv_usec -
                t_start.tv_usec) / 1000.0; \
printf(#name " Time = %f ms.\n",
elapsedTime);

```

2.2 A suitable class for matrices

```

class Matrix
{
private:
    size_t rows;
    size_t cols;
    float * data;
public:
    float * gpu_data;
    size_t len = rows*cols;
    Matrix(): rows(0), cols(0), data(NULL), gpu_data(NULL), len(0) {}
    Matrix(size_t r, size_t c):rows(r), cols(c), len(r * c), data(new float[r*c])
    {
        cudaMalloc(&gpu_data, rows * cols * sizeof(float));
    }
    Matrix(const Matrix &m):rows(m.rows), cols(m.cols), len(m.cols * m.rows), data(new float[m.rows*m
    {
        cudaMalloc(&gpu_data, rows * cols * sizeof(float));
        for(size_t i = 0; i < rows*cols; i++)
            data[i] = m.data[i]; //hard copy
        cudaMemcpy(gpu_data, data, rows * cols * sizeof(float), cudaMemcpyHostToDevice);
    }
    ~Matrix()
    {
        if(data != NULL)
            delete[] data;
        if(gpu_data != NULL)
            cudaFree(gpu_data);
    }
}

```

```

Matrix & operator=(const Matrix &m)
{
    if(this == &m)
        return *this;
    if(data != NULL)
        delete[] data;
    if(gpu_data != NULL)
        cudaFree(gpu_data);
    rows = m.rows;
    cols = m.cols;
    data = new float[rows*cols];
    cudaMalloc(&gpu_data, rows * cols * sizeof(float));
    for(size_t i = 0; i < rows*cols; i++)
        data[i] = m.data[i];
    cudaMemcpy(gpu_data, data, rows * cols * sizeof(float), cudaMemcpyHostToDevice);
    return *this;
}

void set(float val)
{
    for(size_t i = 0; i < rows*cols; i++)
        data[i] = val;
    cudaMemcpy(gpu_data, data, rows * cols * sizeof(float), cudaMemcpyHostToDevice);
}

bool scalar_addcpu(const Matrix &fmat, float factor, float add);
bool scalar_addgpu(const Matrix &fmat, float factor, float add);
Matrix & multiply_openblas(const Matrix &fmat);
Matrix & multiply_cublas(const Matrix &fmat);
void print();
};

void Matrix::print()
{
    printf("Result = [%.1f, ..., %.1f]\n", this->data[0], this->data[this->cols*this->rows-1]);
}

```

There are different constructors and a destructor as well as copy assignment operation. More vitally, there are required functions including the function to set value, the function to print and etc;

2.3 Scalar Arithmetics

CPU Version

```
bool Matrix::scalar_addcpu(const Matrix &fmat, float factor, float add)
{
    if (fmat.rows != this->rows || fmat.cols != this->cols)
    {
        throw "Size mismatch";
    }
    if (fmat.data == NULL || this->data == NULL)
    {
        throw "Empty matrix";
    }
    for(size_t i = 0; i < rows*cols; i++)
        this->data[i] = fmat.data[i] * factor + add;
    cudaMemcpy(this->gpu_data, this->data, len * sizeof(float), cudaMemcpyHostToDevice);

    return true;
}
```

GPU Version

It is important to check parameter. --Prof.Yu

```
__global__ void addKernel(const float * input, float factor, float add, float * output, size_t len)
{
    int i = blockDim.x * blockIdx.x + threadIdx.x;
    if(i < len)
        output[i] = factor*input[i] + add;
}

bool Matrix::scalar_addgpu(const Matrix &fmat, float factor, float add)
{
    if (fmat.rows != this->rows || fmat.cols != this->cols)
        throw "Size mismatch";
    if (fmat.data == NULL || this->data == NULL)
        throw "Empty matrix";
}
```

```

float* d_input = nullptr;
float* d_output = nullptr;

cudaError_t cudaStatus;
cudaStatus = cudaMalloc(&d_input, len * sizeof(float));
if (cudaStatus != cudaSuccess)
    throw "cudaMalloc failed for d_input!";

cudaStatus = cudaMalloc(&d_output, len * sizeof(float));
if (cudaStatus != cudaSuccess)
{
    cudaFree(d_input);
    throw "cudaMalloc failed for d_output!";
}

cudaStatus = cudaMemcpy(d_input, fmat.gpu_data, len * sizeof(float), cudaMemcpyHostToDevice);
if (cudaStatus != cudaSuccess)
{
    cudaFree(d_input);
    cudaFree(d_output);
    throw "cudaMemcpy failed for d_input!";
}

addKernel<<<(len + 255) / 256, 256>>>(d_input, factor, add, d_output, len);

cudaStatus = cudaMemcpy(this->gpu_data, d_output, len * sizeof(float), cudaMemcpyDeviceToHost);
if (cudaStatus != cudaSuccess)
{
    cudaFree(d_input);
    cudaFree(d_output);
    throw "cudaMemcpy failed for d_output!";
}

cudaFree(d_input);
cudaFree(d_output);
cudaMemcpy(this->data, this->gpu_data, len * sizeof(float), cudaMemcpyDeviceToHost);
return true;
}

```

2.4 Matrix Multiplication

OpenBLAS Version

```
Matrix & Matrix::multiply_openblas(const Matrix &fmat)
{
    if (fmat.rows != this->cols)
    {
        throw "Size mismatch";
    }
    if (fmat.data == NULL || this->data == NULL)
    {
        throw "Empty matrix";
    }
    Matrix * rmat = new Matrix(this->rows, fmat.cols);
    cblas_sgemm(CblasRowMajor, CblasNoTrans, CblasNoTrans, this->rows, fmat.cols, this->cols, 1.0, this->data, fmat.data, rmat->data);
    cudaMemcpy(rmat->gpu_data, rmat->data, len * sizeof(float), cudaMemcpyHostToDevice);
    return *rmat;
}
```

cuBLAS Version

```
Matrix & Matrix::multiply_cublas(const Matrix &fmat)
{
    if (fmat.rows != this->cols)
    {
        throw "Size mismatch";
    }
    if (fmat.data == NULL || this->data == NULL)
    {
        throw "Empty matrix";
    }
    Matrix * rmat = new Matrix(this->rows, fmat.cols);
    cublasHandle_t handle;
    cublasCreate(&handle);
    float alpha = 1.0;
    float beta = 0.0;
    cublasSgemm(handle, CUBLAS_OP_N, CUBLAS_OP_N, this->rows, fmat.cols, this->cols, &alpha, this->data, fmat.data, rmat->data);
    cublasDestroy(handle);
    return *rmat;
}
```



```

    cudaMemcpy(rmat->data, rmat->gpu_data, len * sizeof(float), cudaMemcpyDeviceToHost);

    return * rmat;
}

```

In multiplication of two different version, a temporary result matrix was created to store the result and returned to the target result matrix created using copy constructor.

And `cblas_sgemm()` and `cublasSgemm` automatically process all the parameters to multiply and finally we get the result.

Here, two items needs to be emphasized that first, result matrix is used to avoid a overlapping store-multiplication operation that leads to bugs. Second, we have different space for data on CPU and GPU, so we need to be careful to make them consistent.

Part 3 - Result & Verification

3.1 Test main function

To verify the result and validity of my design, we use a following main function to test.

```

int main()
{
    struct timeval t_start, t_end;
    double elapsedTime = 0;

    int dev_count = 0;
    int dev_id = 0;
    cudaGetDeviceCount(&dev_count);
    cudaSetDevice(2);
    cudaGetDevice(&dev_id);
    printf("You have %d cuda devices.\n", dev_count);
    printf("You are using device %d.\n", dev_id);

    size_t rows  = 4096;
    size_t cols  = 4096;
    Matrix matrix_add1 = Matrix(rows, cols);
    Matrix matrix_add2 = Matrix(rows, cols);

    matrix_add1.set(1);
    matrix_add2.set(2);
}

```

```

float a = 5;
float b = 2;

Matrix matrix_addition_result1(rows, cols);
matrix_addition_result1.set(0);

Matrix matrix_addition_result2(rows, cols);
matrix_addition_result2.set(0);

try
{
    matrix_addition_result1.scalar_addcpu(matrix_add1,a,b);
    matrix_addition_result1.print();
    matrix_addition_result2.scalar_addgpu(matrix_add2,a,b);
    matrix_addition_result2.print();
}
catch(const char * msg)
{
    cerr << msg << endl;
}

//test multiplication
matrix_addition_result1.set(1);
matrix_addition_result2.set(1);
matrix_add1.set(1);
matrix_add2.set(1);

try
{
    TIME_START
    Matrix matrix_mul_result1 = matrix_add1.multiply_openblas(matrix_addition_result1);
    TIME_END(OpenBLAS)
    matrix_mul_result1.print();

    TIME_START
    Matrix matrix_mul_result2 = matrix_add2.multiply_cublas(matrix_addition_result2);
    TIME_END(CUBLAS)
    matrix_mul_result2.print();
}
catch(const char * msg)
{
    cerr << msg << endl;
}

return 0;
}

```

Two parts are included which calculate scalar arithmetics and multiplication respectively.

3.2 Result

The result includes result of scalar arithmetics and result of multiplication. Here, we have a matrix of 1 and another matrix of 2 at the beginning. We then multiply a scalar 'a' and add a scalar 'b' so we have the result matrix.

Since $a = 5$ and $b = 2$, result matrices are respectively of entry 7 and entry 12.

```
You have 1 cuda devices.  
You are using device 0.  
Result = [7.0, ..., 7.0]  
Result = [12.0, ..., 12.0]
```

The result is consistent with my deduction.

Running on my PC

CPU parameter:i7-11800H, 8 cores, 16 threads

GPU parameter:RTX 3070

```
● chlorine@DESKTOP-CR08L6M:/mnt/e/C++/project5$ ./a.out  
You have 1 cuda devices.  
You are using device 0.  
Result = [7.0, ..., 7.0]  
Result = [12.0, ..., 12.0]  
OpenBLAS Time = 957.655000 ms.  
Result = [4096.0, ..., 4096.0]  
CUBLAS Time = 476.869000 ms.  
Result = [4096.0, ..., 4096.0]
```

After waking up my computer and test for several times, we observed that time cost of OpenBLAS varies around 950ms while cuBLAS time cost is more stable around 480ms.

Running on the Server

```
$ ./a.out  
You have 4 cuda devices.  
You are using device 2.  
Result = [7.0, ..., 7.0]  
Result = [12.0, ..., 12.0]  
OpenBLAS Time = 134.706000 ms.  
CUBLAS Time = 61.897000 ms.  
$
```

In contrast, running the same code on the server, the OpenBLAS time cost is reduced to around 110ms and cuBLAS is reduced to around 60ms.

By the result, conclusion can be derived that cores of CPU influence the time cost of OpenBLAS while numbers of devices influence the time cost of cuBLAS, while cuBLAS is even faster.

To summarize, GPU devices with CUDA indeed save a lot of time cost to make codes more efficiently, which is important for the whole AI industry.

Part 4 - Difficulty & Solution

1. To complete this project, we have to learn a lot rules embedded in CUDA toolkit and we need to understand meanings of each part during the calculation using kernel(For example, no kernel in the class).
2. During testing my code, there might be some errors struggling and troubling, so I use some try-catch operation to assist me check the bugs. It is newly learned but really helpful in this project.
3. Don't forget to link the cuBLAS and OpenBLAS library when compile the source code.

Part 5 Appendix

experience, comments and reference

5.1 Experience and Comments

Audience may found that I have less content for Part 4 because it is really not a big project and I think I've managed those basic grammar in C/C++. Another reason will be actually I still encounter many problems but either I forgot how it is done or I would like to introduce those stupid mistakes here because I think that is no difficulty but carelessness.

Initially, I found the member function `print()` failed to show the result of GPU result. For multiple times I checked my source code but ended up in finding nothing. Suddenly, I realized `print()` only showed data in CPU cache but nothing in GPU, because I had

forgot to make content of `cpu_data` and `gpu_data` of the matrix consistent. Therefore, I appended a `memcpy` operation for each function to make them consistent.

Then, I have to apologize for that at very first time I used to be tired of this project because I've almost been devastated by the previous projects. As a result, I did not test the accuracy of multiplication at first but soon it occurred to me that something went wrong.

Therefore, I had to be concentrated, careful, enjoyable, and energetic again to fix this problem. It turned out to be a result of returning `this object` while doing multiplication. In another word, I fix the data of matrix while doing multiplication. It is really stupid and I hope that I would no longer do something like this...

5.2 Reference

NVIDIA CUDA [CUDA 编程手册系列](#)

CSDN Blog [CUDA入门——cudaMalloc函数的理解](#)

CSDN Blog [在类内调用cuda内核函数，出现的错误：illegal combination of memory qualifiers 等问题的解决](#)

Linux-Console.net [如何在 Ubuntu 20.04 LTS 上安装 CUDA](#)